Reduced-Complexity Key Equation Solvers for Generalized Integrated Interleaved BCH Decoders

Zhenshan Xie and Xinmiao Zhang, *Senior Member, IEEE*

***Abstract*— Generalized integrated interleaved (GII) codes nest linear block codewords to generate codewords belonging to stronger linear block codes. They can achieve very high throughput with excellent error-correcting capability. GII codes can be based on either Reed-Solomon (RS) or BCH codes. GII-BCH codes are the most promising candidate for error correction in next-generation terabit/s Flash and storage class memories, and optical communications. On the other hand, GII decoder implementation faces many challenges. In particular, the key equation solver (KES) in the nested decoding process causes not only clock frequency bottleneck but also large area. Although techniques have been developed recently to address these issues for GII-RS codes, they cannot be directly extended for binary GII-BCH codes if every other iteration of the nested KES is skipped to reduce the latency. Two instead of one higher-order syndromes need to be incorporated into each nested KES iteration and this makes the implementation much more challenging. In this paper, algorithmic reformulations and architectural modifications are developed to eliminate the clock frequency bottleneck and reduce the area of GII-BCH nested KES. Additionally, the number of processing elements is substantially reduced by analyzing the minimum number of coefficients to keep for the involved polynomials without undesirable degradation on the error-correcting performance. The critical path of the proposed scaled nested BCH KES architecture with reduced processing elements is only one multiplier and three adders. For an example GII-BCH code over $GF(2^{12})$ that has a $t_v = 58$-error-correcting code as the most powerful code generated by the nesting, our design also further reduces the area by 33%.**

***Index Terms*— BCH codes, error-correcting codes, generalized integrated interleaved codes, key equation solver, VLSI architectures.**

I. Introduction

GENERALIZED integrated interleaved (GII) codes [1] nest a set of codewords of a linear block code to generate codewords of stronger linear block codes. The linear block codes considered are typically Reed-Solomon (RS) or BCH codes. Most of the time, the errors are corrected by decoding individual sub-codewords and throughput in the order of hundreds of Gigabit/s can be easily reached. Besides, more errors are correctable by utilizing the nested codewords. Although low-density parity-check (LDPC) and Polar codes can achieve higher coding gain when soft channel inputs are available, they cannot achieve such high throughput and good tradeoff on error-correcting performance. GII codes are the answer to the hyper-speed and good error-correction capability requirements posed by next-generation digital communications and storage. Particularly, BCH codes are used in Flash memories, storage class memories, and optical communications. As the technology scales down, channel deteriorates, and/or throughput requirement goes up, BCH-based GII codes are the best replacement of conventional BCH codes for these systems.

The 2-layer GII nesting in [1] has been modified in [2] and extended to 3-layer codes in [3] to further improve the decoding locality. The procedure and architecture for GII encoding can be found in [4], [5]. This paper focuses on GII-BCH decoder design. Regardless of the nesting scheme, the GII decoding has two stages [5], [6]. The first stage is conventional BCH decoding on individual sub-codewords, in which a key equation solver (KES), such as the Berlekamp-Massey (BM) algorithm [7], utilizes syndromes to iteratively find an error locator polynomial. If there are extra errors, the second-stage nested decoding is activated. The higher-order syndromes computed from the stronger nested codewords are used by the KES to find a longer error locator polynomial and accordingly correct more errors in the sub-codewords. The KES step limits the achievable clock frequency of the overall decoder and requires large silicon area to implement.

Each iteration of the KES first computes a discrepancy coefficient, which is then used to update the error locator polynomial. Traditionally, the discrepancy coefficient is calculated using a large multiplier-adder tree. Besides, another multiplier and an adder are in the data path for polynomial updating. This long data path limits the achievable clock frequency. Alternatively, the reformulated inversionless Berlekamp-Massey (riBM) architecture [8] initializes a discrepancy polynomial using the syndromes and updates this polynomial simultaneously with the error locator polynomial. As a result, the critical path is reduced to one multiplier and one adder.

The KES for the nested decoding does not have to restart. It can continue from the results of the KES for the first-stage sub-codeword decoding, and more iterations are carried out to incorporate the higher-order syndromes [5]. However, the high-order syndromes are not available in the very beginning to initialize the discrepancy polynomial and the riBM architecture cannot be applied to the nested KES. It was proposed in [9] to re-initialize the discrepancy polynomial at the beginning of the nested KES. However, such re-initialization needs many multiplier-adder trees, which lead

Manuscript received January 28, 2020; revised April 14, 2020 and May 8, 2020; accepted May 30, 2020. Date of publication June 10, 2020; date of current version December 1, 2020. This work was supported in part by Kioxia Corporation. This article was recommended by Associate Editor P. K. Meher. *(Corresponding author: Zhenshan Xie.)*

The authors are with the Department of Electrical and Computer Engineering, The Ohio State University, Columbus, OH 43210 USA (e-mail: xie.855@osu.edu).

Color versions of one or more of the figures in this article are available online at https://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TCSI.2020.2999823

1549-8328 © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 29,2026 at 14:11:03 UTC from IEEE Xplore. Restrictions apply.

to large area and long critical path. In [10], the higher-order syndromes are incorporated one by one into the discrepancy polynomial for lower-order syndromes, and the resulted area is much smaller. Besides, after applying the slow-down and re-timing techniques [11], the critical path is reduced to one multiplier and one adder. The area of such a nested KES architecture is further reduced by scaling the polynomials to enable the sharing of product terms [12].

The designs in [10], [12] are targeting at GII-RS codes. In the decoding of conventional binary BCH codes, the odd iterations of the KES can be skipped, and the computations in the even iterations have similar format and complexity as those in the original algorithm [13]. The odd iterations of the nested KES for GII-BCH decoding can be also skipped. However, unlike that in the KES for traditional BCH decoding, the odd syndromes affect the nested KES computations and they still need to be incorporated even if the odd iterations are skipped. As a result, two instead of one higher-order syndromes need to be incorporated in each even iteration of the nested KES to derive the discrepancy coefficient if the odd iterations are skipped. This makes the polynomial updating formulas more complicated and the results in [10] for GII-RS codes cannot be directly extended for nested BCH KES. Additionally, the more complicated polynomial updating makes it much more challenging to scale the polynomials in order to enable the sharing of product terms for area reduction. Extending the scaling schemes in [12] would lead to two multipliers in the critical path, and new scaling schemes are necessary to reduce the complexity of the nested BCH KES while keeping only one multiplier in the critical path.

This paper proposes algorithmic reformulations and architectural modifications to eliminate the clock frequency bottleneck and reduce the area of the nested BCH KES when the odd iterations are skipped. Three designs are developed in this paper.

i) Through analyzing the possible computations to be carried out in odd iterations and combining them with those in even iterations, a nested BCH KES algorithm is developed to skip the odd iterations and compute the discrepancy coefficients without using a multiplier-adder tree. Accordingly, the critical path can be reduced to just one multiplier and two adders by applying the slow-down and re-timing techniques.

ii) A scaled nested KES algorithm and corresponding architecture are developed for GII-BCH codes to enable the sharing of product terms in polynomial updating. As a result, the number of multipliers required is substantially reduced. Additionally, by reformulating the scalar computations, the critical path is kept at one multiplier and three adders.

iii) By analyzing the minimum number of coefficients to keep for the polynomials involved in the nested KES without undesirable degradation on the decoding error-correcting performance, the number of processing elements is substantially reduced. This technique also applies to nested RS KES.

The clock frequency bottleneck of the nested BCH KES with odd iterations skipped is eliminated in all the three proposed designs. Additionally, compared to design i), substantial area reduction is achievable by designs ii) and iii) through reducing the complexity and number, respectively, of processing elements. For an example GII-BCH code over $GF(2^{12})$ whose most powerful nested codeword is able to correct up to $t_\upsilon = 58$ errors, 33% area reduction is achieved by designs ii) and iii) compared to design i).

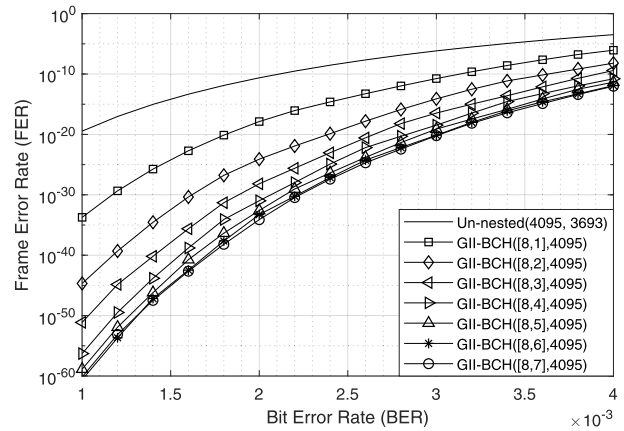


Fig. 1. FERs of GII-BCH codes over $GF(2^{12})$ with $n = 4095$, $m = 8$, and 90.2% code rate.

This paper is organized as follows. Section II introduces the decoding of GII-BCH codes. The three proposed nested BCH KES designs are detailed in Section III. Section IV provides complexity analyses and comparisons, and conclusions follow in Section V.

II. GII-BCH Codes and Decoding

Let $\mathcal{C}_\upsilon \subseteq \mathcal{C}_{\upsilon-1} \subseteq \cdots \subseteq \mathcal{C}_1 \subset \mathcal{C}_0$ be $\upsilon + 1$ BCH codes of length n and dimension $k_\upsilon \leq k_{\upsilon-1} \leq \cdots \leq k_1 < k_0$ over $GF(2^q)$. The GII code is defined as [1], [5]

$$\mathcal{C} \triangleq \left\{ c = [c_0, \cdots, c_{m-1}] : c_i \in C_0, \tilde{c}_l = \sum_{i=0}^{m-1} \alpha^{il} c_i \in \mathcal{C}_{\upsilon-l}, 0 \leq l < \upsilon \right\}, \quad (1)$$

where α is a primitive element of GF(2^q). Basically, a GII codeword consists of m sub-codewords $c_0, c_1, \cdots, c_{m-1} \in \mathcal{C}_0$, and the nesting of the m sub-codewords defined as linear combinations generates υ codewords $\tilde{c}_l \in \mathcal{C}_{\upsilon-l}$ that have higher correction capability. Each $\mathcal{C}_i$ can be also a RS code.

The GII-BCH decoding has two stages. The first stage is the traditional BCH decoding of individual sub-codewords $c_0, c_1, \cdots, c_{m-1}$. Let the error-correcting capabilities of $\mathcal{C}_\upsilon, \cdots \mathcal{C}_1, \mathcal{C}_0$ be $t_\upsilon \geq \cdots \geq t_1 > t_0$. When the number of errors in any sub-codeword exceeds t_0, the second-stage nested decoding is activated. Sort the number of errors in the sub-codewords as $\tau_0 \leq \tau_1 \leq \cdots \leq \tau_{m-1}$. The errors are correctable by the GII code if $\tau_{m-\upsilon-1} \leq t_0$ and $\tau_{m-\upsilon-1+i} \leq t_i$ for $1 \leq i \leq \upsilon$ [1], [5]. Fig. 1 shows the frame error rates (FERs) of GII-BCH codes with $n = 4095$, $m = 8$ and various υ for different input bit error rates (BERs) assuming the binary symmetric channel. All the codes compared have the same rate as 90.2%. For comparison, the FER of 8 un-nested BCH codewords with the same length and redundancy is also included in Fig. 1. It can be observed that the GII-BCH codes can achieve orders of magnitude lower FERs compared to un-nested traditional BCH codes and the performance improves with υ.

First consider traditional BCH decoding. A codeword vector is represented interchangeably as a polynomial in this paper. Denote a t_0-error-correcting BCH codeword by $c(x)$ and the

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 29,2026 at 14:11:03 UTC from IEEE Xplore. Restrictions apply.

Algorithm A riBM Algorithm Simplified for BCH Decoding

input: S_i $(0 \le i < 2t_0)$
initialization:
$\Lambda^{(0)}(x) = 1$; $B^{(0)}(x) = x^{-1}$; $k^{(0)} = -1$; $\gamma^{(0)} = 1$
$\hat{\Delta}^{(0)}(x) = \sum_{i=0}^{2t_0-1} S_i x^i$; $\hat{\Theta}^{(0)}(x) = \sum_{i=1}^{2t_0-1} S_i x^{i-1}$
for $r = 0, 2, \cdots, 2t_0 - 2$
1). $\Lambda^{(r+2)}(x) = \gamma^{(r)}\Lambda^{(r)}(x) + \hat{\Delta}_0^{(r)} x^2 B^{(r)}(x)$
2). $\hat{\Delta}^{(r+2)}(x) = \gamma^{(r)}\hat{\Delta}^{(r)}(x)/x^2 + \hat{\Delta}_0^{(r)}\hat{\Theta}^{(r)}(x)$
if $\hat{\Delta}_0^{(r)} \neq 0$ and $k^{(r)} \ge -1$
3). $B^{(r+2)}(x) = \Lambda^{(r)}(x)$; $\hat{\Theta}^{(r+2)}(x) = \hat{\Delta}^{(r)}(x)/x^2$
4). $\gamma^{(r+2)} = \hat{\Delta}_0^{(r)}$; $k^{(r+2)} = -k^{(r)} - 2$
else
5). $B^{(r+2)}(x) = x^2 B^{(r)}(x)$; $\hat{\Theta}^{(r+2)}(x) = \hat{\Theta}^{(r)}(x)$
6). $\gamma^{(r+2)} = \gamma^{(r)}$; $k^{(r+2)} = k^{(r)} + 2$

corresponding received word is $y(x) = c(x) + e(x)$, where $e(x)$ is the error polynomial. The decoding starts with computing syndromes $S_j = y(\alpha^{j+1}) = e(\alpha^{j+1})$ $(0 \le j < 2t_0)$. If all $2t_0$ syndromes are zero, there is no error. Otherwise, a KES, such as the BM algorithm [7], can be applied to iteratively find an error locator polynomial $\Lambda(x)$ and an error evaluator polynomial $\Omega(x)$ satisfying $\Omega(x) \equiv \Lambda(x)S(x) \bmod x^{2t_0}$, where $S(x) = \sum_{j=0}^{2t_0-1} S_j x^j$. The BM algorithm initializes $\Lambda^{(0)}(x) = 1$. In the r-th iteration, it first computes a discrepancy coefficient

$$\delta^{(r)} = \sum_{i=0}^{L_\Lambda^{(r)}} \Lambda_i^{(r)} S_{r-i}, \tag{2}$$

where $\Lambda_i^{(r)}$ is the i-th coefficient of $\Lambda^{(r)}(x)$ and $L_\Lambda^{(r)}$ is the length of $\Lambda^{(r)}(x)$. Then $\Lambda^{(r)}(x)$ is updated using $\delta^{(r)}$. This process incorporates one more syndrome in each iteration and $2t_0$ iterations are carried out. If the degree of $\Lambda^{(2t_0)}(x)$ equals the number of its distinct roots, which can be told by the Chien search, then the decoding is considered successful and the error locations are the inverse roots.

For binary BCH codes, $S_{2i+1} = S_i^2$. Hence the discrepancies at odd iterations of the BM algorithm are zero and these iterations can be skipped. This simplified algorithm with skipped iterations is referred to as the Berlekamp's algorithm. The large multiplier-adder tree for computing $\delta^{(r)}$ and the dependency of the polynomial updating on $\delta^{(r)}$ lead to two multipliers and $\lceil \log_2(t_0 + 1) \rceil + 1$ adders in the critical path of the KES. This long critical path limits the achievable clock frequency of the overall decoder. $\delta^{(r)}$ can be interpreted as the r-th coefficient of $\Delta(x) = \Lambda^{(r)}(x)S(x)$. To reduce the critical path of the KES, the riBM algorithm [8] initializes a discrepancy polynomial $\hat{\Delta}^{(0)}(x) = \Lambda^{(0)}(x)S(x) = S(x)$ and updates it simultaneously with $\Lambda(x)$. Actually, $\hat{\Delta}^{(r)}(x) = \Lambda^{(r)}(x)S(x)/x^r$, and hence $\delta^{(r)} = \hat{\Delta}_0^{(r)}$. As a result, the critical path is reduced to one multiplier and one adder. Taking advantage of the property that the odd iterations for binary BCH decoding can be skipped, the riBM algorithm can be simplified as in Algorithm A [13]. This algorithm can be implemented by very regular architectures consisting of $3t + 1$ processing elements (PEs), each of which has two multipliers, two registers, one adder, and one multiplexer [8].

To facilitate the separation of the even and odd coefficients in the implementation, the $B(x)$ and $\hat{\Theta}(x)$ in Algorithm A are those in the riBM algorithm of [8] divided by x. The divisions are also reflected in the initialization of these two polynomials in Algorithm A. It can be observed that the odd coefficients of $\hat{\Delta}^{(r)}(x)$ and $\hat{\Theta}^{(r)}(x)$ are not involved in the computation of discrepancy $\hat{\Delta}_0^{(r+2)}$ for the next iteration. In fact, all odd coefficients of $\hat{\Delta}^{(r)}(x)$ and $\hat{\Theta}^{(r)}(x)$ do not interact with the even coefficients of these two polynomials during the iterations. Hence, the odd coefficients of $\hat{\Delta}^{(r)}(x)$ and $\hat{\Theta}^{(r)}(x)$ can be removed, and the initialization of Algorithm A can be changed to $\hat{\Delta}^{(0)}(x) = S_0 + S_2 x^2 + \cdots + S_{2t_0-2} x^{2t_0-2}$ and $\hat{\Theta}^{(0)}(x) = S_1 + S_3 x^2 + \cdots + S_{2t_0-1} x^{2t_0-2}$.

The second-stage nested decoding is more challenging. $2t$ syndromes are needed to correct t errors. When a sub-codeword has more than t_0 errors, more syndromes are needed to correct them. However, the higher-order syndromes cannot be computed from the sub-codeword itself. Instead, the nested codewords belong to more powerful codes and more syndromes can be computed from them as $\tilde{S}_j^{(l)} = \tilde{y}_l(\alpha^{j+1})$ $(0 \le l < \upsilon, 0 \le j < 2t_{\upsilon-l})$, where $\tilde{y}_l(x) = \sum_{i=0}^{m-1} \alpha^{il} y_i(x)$ and $y_i(x)$ is the i-th received sub-codeword. Syndromes of the nested codewords and those of the sub-codewords are connected by the same nesting as defined in (1). Let the indices of the $b \le \upsilon$ sub-codewords with extra errors be $i_1, i_2, \cdots, i_b$. Since $\tilde{c}_0, \tilde{c}_1, \cdots, \tilde{c}_{b-1}$ are at least t_1-error-correcting, $2t_1$ syndromes can be computed from each of them. Accordingly, the $2t_0 \le j < 2t_1$ higher-order syndromes of the sub-codewords with extra errors can be computed by

$$\left[S_j^{(i_1)},\ S_j^{(i_2)}, \cdots, S_j^{(i_b)}\right]^T = A^{-1}\left[\tilde{S}_j^{(0)},\ \tilde{S}_j^{(1)}, \cdots, \tilde{S}_j^{(b-1)}\right]^T, \tag{3}$$

where 'T' denotes transpose and the entry of A in the p-th row and q-th column is $\alpha^{p i_{q+1}}$ $(0 \le p, q < b)$ according to the nesting specified in (1). After the sub-codeword with up to t_1 errors is corrected using the $2t_1$ syndromes, its corresponding entry and column are deleted from the syndrome vector and A, respectively, in (3). Besides, the last row of A and the last entry in the nested syndrome vector are also deleted. $\tilde{c}_0, \tilde{c}_1, \cdots, \tilde{c}_{b-2}$ are at least t_2-error-correcting. Hence, $2t_1 \le j < 2t_2$ higher-order syndromes for the remaining sub-codewords can be computed from (3) with updated A and syndrome vectors to correct more errors. This process is repeated for up to $b \le \upsilon$ rounds.

The KES for the nested decoding can continue from the results of the KES for individual sub-codeword decoding [5]. More iterations are carried out to incorporate the higher-order syndromes. However, the higher-order syndromes are not available at the very beginning of the KES to initialize the discrepancy polynomial and derive the discrepancy coefficients in the same manner as in the riBM algorithm. Computing the discrepancy coefficients using (2) and updating the polynomials afterwards in the nested KES iterations again lead to long critical path. To address this issue for GII-RS decoding, the discrepancy coefficient computation for the nested KES iterations is reformulated and the higher-order syndromes are incorporated one by one into the discrepancy polynomial from the KES of the sub-codeword decoding. Additionally, the slow-down and re-timing techniques [11] were applied

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 29,2026 at 14:11:03 UTC from IEEE Xplore. Restrictions apply.

to reduce the critical path to one multiplier and one adder. The nested RS KES complexity was further reduced by using scaled versions of the involved polynomials to enable the sharing of product terms in polynomial updating [12].

Despite the work on the nested KES for RS codes, the nested KES for BCH codes has many challenges. If we want to skip the odd iterations, two higher-order syndromes need to be integrated into the discrepancy coefficient computation at a time. As a result, the reformulation of the higher-order discrepancy coefficient computation and corresponding polynomial updating become more complicated and the computations no longer remain similar to those in the nested RS KES. Additionally, due to the more complex polynomial updating, scaling the polynomials for product term sharing becomes more challenging. Also incorporating two higher-order syndromes at a time introduces one more multiplier in the critical path for scalar computations. Moreover, the higher coefficients of the discrepancy polynomial may not affect later nested KES iterations in the last nested decoding round. This needs to be studied in order to eliminate the higher coefficients and accordingly further reduce the nested KES hardware complexity. All these issues are addressed in our designs presented in the next section.

Although the simplified inversionless Berlekamp-Massey (SiBM) architecture in [14] reduces the number of PEs from $3t+1$ as in the riBM architecture to $2t+1$ for conventional binary BCH decoding, it cannot be adopted for the nested KES. The SiBM architecture utilizes the property that the error magnitudes do not need to be computed for binary BCH decoding. Accordingly, the higher coefficients of $\hat{\Delta}(x)$ are overwritten with the coefficients of $\Lambda(x)$ to reduce the number of PEs. However, the higher coefficients of $\hat{\Delta}(x)$ are also required to compute the discrepancy coefficients in later nested decoding rounds when higher-order syndromes are incorporated. Hence they need to be kept for the nested KES.

III. Nested KES for GII-BCH Decoding

This section proposes three nested KES designs for GII-BCH decoding. Each design improves upon the previous one. First, the discrepancy coefficient computation and polynomial updating are reformulated to eliminate the clock frequency bottleneck in the case that every other nested KES iteration is skipped. The resulted algorithm can be also implemented by a regular architecture consisting of identical PEs. Then the complexity of the PEs is lowered in the second design by our scaled nested BCH KES algorithm and architecture that enable the sharing of product terms without increasing the number of multipliers in the critical path. In the third design, the number of PEs is reduced by eliminating higher polynomial coefficients with negligible performance loss.

A. Nested BCH KES With Eliminated Clock Frequency Bottleneck

When a BCH sub-codeword has more than t_0 errors, higher-order syndromes $S_{2t_0}, S_{2t_0+1}, \cdots$ are obtained from the nested codewords as mentioned previously. $S(x) = S_0 + S_1x + \cdots + S_{2t_0-1}x^{2t_0-1}$ is the syndrome polynomial used in the sub-codeword decoding. The syndrome polynomial including the higher-order syndromes can be represented by $S'(x) = S(x) + S_{2t_0}x^{2t_0} + S_{2t_0+1}x^{2t_0+1} + \cdots$. According to the definition in (2), $\delta^{(r)}$ for $r = 2t_0, 2t_0+1, \cdots$ is still the r-th coefficient of $\Delta'^{(r)}(x) = \Lambda^{(r)}(x)S'(x)$. From the KES for sub-codeword decoding, $\Lambda^{(2t_0)}(x)$ and $\hat{\Delta}^{(2t_0)}(x)$ are available and $x^{2t_0}\hat{\Delta}^{(2t_0)}(x) = \Delta^{(2t_0)}(x) = \Lambda^{(2t_0)}(x)S(x)$. Hence, as it has been derived in [10], the coefficients of $\Delta'^{(2t_0)}(x)$ are connected to those of $\Lambda^{(2t_0)}(x)$ and $\hat{\Delta}^{(2t_0)}(x)$ as follows

$$\begin{aligned}
\Delta'^{(2t_0)}_{2t_0} &= (\hat{\Delta}^{(2t_0)}_0 + \Lambda^{(2t_0)}_0 S_{2t_0}) \\
\Delta'^{(2t_0)}_{2t_0+1} &= (\hat{\Delta}^{(2t_0)}_1 + \Lambda^{(2t_0)}_1 S_{2t_0}) + \Lambda^{(2t_0)}_0 S_{2t_0+1} \\
\Delta'^{(2t_0)}_{2t_0+2} &= (\hat{\Delta}^{(2t_0)}_2 + \Lambda^{(2t_0)}_2 S_{2t_0}) + \Lambda^{(2t_0)}_1 S_{2t_0+1} + \Lambda^{(2t_0)}_0 S_{2t_0+2} \\
&\vdots
\end{aligned} \tag{4}$$

Only one discrepancy coefficient is needed in each nested KES iteration. Accordingly, as it can be observed from (4), the higher-order syndromes can be incorporated into the iterations one at a time, starting from S_{2t_0}. Assume that the polynomials are initialized as in Algorithm A in the sub-codeword decoding. Utilizing (4) and the polynomial updating in the riBM algorithm [8], it has been derived in [10] that the $\Lambda(x)$ and $\hat{\Delta}(x)$ for the nested RS KES can be updated as follows.

$$\begin{aligned}
\Lambda^{(r+1)}(x) &= \gamma^{(r)}\Lambda^{(r)}(x) + \hat{\Delta}^{(r)}_0 x^2 B^{(r)}(x) \\
\hat{\Delta}^{(r+1)}(x) &= \gamma^{(r)}(\hat{\Delta}^{(r)}(x)/x + S_{r+1}\Lambda^{(r)}(x)) \\
&\quad + \hat{\Delta}^{(r)}_0(x\hat{\Theta}^{(r)}(x) + S_{r+1}x^2B^{(r)}(x))
\end{aligned} \tag{5}$$

Different from GII-RS decoding, it is known that the discrepancy coefficients in the odd iterations $r+1 = 2t_0+1$, $2t_0+3, \cdots$ of the KES are zero in binary GII-BCH decoding. From (5), when $\hat{\Delta}^{(r+1)}_0 = 0$, it can be derived that

$$\begin{aligned}
\Lambda^{(r+2)}(x) &= \Lambda^{(r+1)}(x) \\
\hat{\Delta}^{(r+2)}(x) &= \hat{\Delta}^{(r+1)}(x)/x + S_{r+2}\Lambda^{(r+1)}(x)
\end{aligned} \tag{6}$$

The multiplications by $\gamma^{(r+1)}$ are omitted from both polynomial updating since scaling all polynomials by the same nonzero factor does not change the decoding results. Substituting (5) into (6), the polynomial updating for both iterations can be combined as

$$\begin{aligned}
\Lambda^{(r+2)}(x) &= \gamma^{(r)}\Lambda^{(r)}(x) + \hat{\Delta}^{(r)}_0 x^2 B^{(r)}(x) \\
\hat{\Delta}^{(r+2)}(x) &= \gamma^{(r)}(\hat{\Delta}^{(r)}(x)/x^2 + S_{r+1}\Lambda^{(r)}(x)/x + S_{r+2}\Lambda^{(r)}(x)) \\
&\quad + \hat{\Delta}^{(r)}_0(\hat{\Theta}^{(r)}(x) + S_{r+1}xB^{(r)}(x) + S_{r+2}x^2B^{(r)}(x))
\end{aligned} \tag{7}$$

The updating formulas of $B^{(r+2)}(x)$ and $\hat{\Theta}^{(r+2)}(x)$ for nested BCH KES can be also derived by combining the computations for two iterations taking into account that $\hat{\Delta}^{(r+1)}_0 = 0$. In the nested KES for RS codes [10], if $B^{(0)}(x)$ was initialized to x^{-1}, $B^{(r+1)}(x)$ is updated as $\Lambda^{(r)}(x)/x$ when $(\hat{\Delta}^{(r)}_0 \neq 0)\&(k^{(r)} \geq -1)$ or $xB^{(r)}(x)$ otherwise in iteration r. When $\hat{\Delta}^{(r+1)}_0 = 0$, $B^{(r+2)}(x) = xB^{(r+1)}(x)$. As a result,

$$B^{(r+2)}(x) = \begin{cases} \Lambda^{(r)}(x), & \text{if } (\hat{\Delta}^{(r)}_0 \neq 0)\&(k^{(r)} \geq -1) \\ x^2B^{(r)}(x), & \text{otherwise.} \end{cases} \tag{8}$$

From [10], $\hat{\Theta}^{(r+1)}(x)$ equals $\hat{\Delta}^{(r)}(x)/x^2 + S_{r+1}\Lambda^{(r)}(x)/x$ if $(\hat{\Delta}^{(r)}_0 \neq 0)\&(k^{(r)} \geq -1)$ or $\hat{\Theta}^{(r)}(x) + S_{r+1}xB^{(r)}(x)$ otherwise. For the first case, substituting $\hat{\Delta}^{(r+1)}_0 = 0$

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 29,2026 at 14:11:03 UTC from IEEE Xplore. Restrictions apply.

Algorithm B Nested GII-BCH KES Algorithm

input: $\Lambda^{(u)}_{even}(x)$, $\Lambda^{(u)}_{odd}(x)$, $B^{(u)}_{even}(x)$, $B^{(u)}_{odd}(x)$, $\hat{\Delta}^{(u)}_{even}(x)$, $\hat{\Theta}^{(u)}_{even}(x)$, $\gamma^{(u)}$, $k^{(u)}$ from previous KES; S_i $(u \le i < w)$; $S_w = 0$

initialization:

$\hat{\Delta}^{(u)}_{even}(x) = \hat{\Delta}^{(u)}_{even}(x) + S_u \Lambda^{(u)}_{even}(x)$

$\hat{\Theta}^{(u)}_{even}(x) = \hat{\Theta}^{(u)}_{even}(x) + S_u B^{(u)}_{even}(x)$

for $r = u, u+2, \cdots, w-2$

1) $\Lambda^{(r+2)}_{even}(x) = \gamma^{(r)} \Lambda^{(r)}_{even}(x) + \hat{\Delta}^{(r)}_0 x^2 B^{(r)}_{even}(x)$

2) $\Lambda^{(r+2)}_{odd}(x) = \gamma^{(r)} \Lambda^{(r)}_{odd}(x) + \hat{\Delta}^{(r)}_0 x^2 B^{(r)}_{odd}(x)$

3) $\hat{\Delta}^{(r+2)}_{even}(x) = \gamma^{(r)} (\hat{\Delta}^{(r)}_{even}(x)/x^2 + S_{r+1} \Lambda^{(r)}_{odd}(x)/x + S_{r+2} \Lambda^{(r)}_{even}(x)) + \hat{\Delta}^{(r)}_0 (\hat{\Theta}^{(r)}_{even}(x) + S_{r+1} x B^{(r)}_{odd}(x) + S_{r+2} x^2 B^{(r)}_{even}(x))$

if $\hat{\Delta}^{(r)}_0 \neq 0$ and $k^{(r)} \ge -1$

4) $B^{(r+2)}_{even}(x) = \Lambda^{(r)}_{even}(x)$
$B^{(r+2)}_{odd}(x) = \Lambda^{(r)}_{odd}(x)$

5) $\hat{\Theta}^{(r+2)}_{even}(x) = \hat{\Delta}^{(r)}_{even}(x)/x^2 + S_{r+1} \Lambda^{(r)}_{odd}(x)/x + S_{r+2} \Lambda^{(r)}_{even}(x)$

6) $\gamma^{(r+2)} = \hat{\Delta}^{(r)}_0$; $k^{(r+2)} = -k^{(r)} - 2$

else

7) $B^{(r+2)}_{even}(x) = x^2 B^{(r)}_{even}(x)$
$B^{(r+2)}_{odd}(x) = x^2 B^{(r)}_{odd}(x)$

8) $\hat{\Theta}^{(r+2)}_{even}(x) = \hat{\Theta}^{(r)}_{even}(x) + S_{r+1} x B^{(r)}_{odd}(x) + S_{r+2} x^2 B^{(r)}_{even}(x)$

9) $\gamma^{(r+2)} = \gamma^{(r)}$; $k^{(r+2)} = k^{(r)} + 2$

into the formula, it can be derived that $\hat{\Theta}^{(r+2)}(x) = \hat{\Theta}^{(r+1)}(x) + S_{r+2} x B^{(r+1)}(x) = \hat{\Delta}^{(r)}(x)/x^2 + S_{r+1} \Lambda^{(r)}(x)/x + S_{r+2} x B^{(r+1)}(x)$. If the odd iterations are skipped, $B^{(r+1)}(x)$ is not available. Fortunately, $S_{r+2} x B^{(r+1)}(x)$ can be also written as $S_{r+2} \Lambda^{(r)}(x)$ when $(\hat{\Delta}^{(r)}_0 \neq 0) \& (k^{(r)} \ge -1)$. The updating formula for the second case can be also derived. Overall,

$$\hat{\Theta}^{(r+2)}(x) = \begin{cases} \hat{\Delta}^{(r)}(x)/x^2 + S_{r+1} \Lambda^{(r)}(x)/x + S_{r+2} \Lambda^{(r)}(x), \\ \quad \text{if } (\hat{\Delta}^{(r)}_0 \neq 0) \& (k^{(r)} \ge -1) \\ \hat{\Theta}^{(r)}(x) + S_{r+1} x B^{(r)}(x) + S_{r+2} x^2 \ B^{(r)}(x), \\ \quad \text{otherwise.} \end{cases} \tag{9}$$

All coefficients of $\Lambda(x)$ and $B(x)$ need to be kept. On the other hand, as mentioned previously, the odd coefficients of $\hat{\Delta}(x)$ and $\hat{\Theta}(x)$ do not affect $\hat{\Delta}^{(r)}_0$ in any of the nested KES iterations for BCH decoding and hence do not need to be computed or stored. Decompose $\Lambda(x)$ into $\Lambda_{even}(x) = \Lambda_0 + \Lambda_2 x^2 + \cdots$ and $\Lambda_{odd}(x) = \Lambda_1 x + \Lambda_3 x^3 + \cdots$ and decompose the other polynomials in a similar way. The nested KES for incorporating higher-order syndromes $S_u, S_{u+1}, \cdots S_{w-1}$ into the KES results from the sub-codeword decoding or previous nested decoding round covering syndromes up to S_{u-1} for GII-BCH decoding can be carried out according to Algorithm B.

The initialization of $\hat{\Delta}^{(u)}_{even}(x)$ and $\hat{\Theta}^{(u)}_{even}(x)$ in Algorithm B is necessary to incorporate the first higher-order syndrome, S_u, to derive the discrepancy coefficient $\hat{\Delta}^{(u)}_0$ according to the first equation in (4). The updating of the even and odd coefficients of the polynomials can be directly derived from (7), (8), and (9). In the last iteration of $r = w-2$, S_w is needed as S_{r+2}.

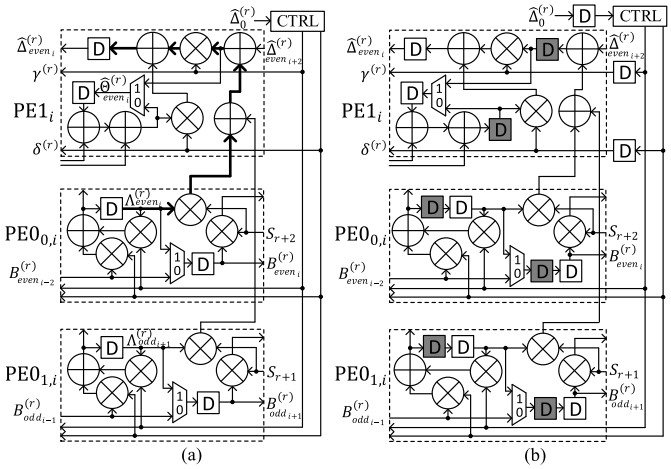


Fig. 2. GII-BCH nested KES PE architectures. (a) original architecture; (b) slowed-down and re-timed architecture.

It is not one of the higher-order syndromes available in the current nested decoding round and is set to 0 in Algorithm B. As a result, the $S_{r+2} x^2 B^{(r)}_{even}(x)$ and $S_{r+2} \Lambda^{(r)}_{even}(x)$ terms are not added to the corresponding polynomials in the last iteration. However, these terms will be taken care of by the initialization in the next nested decoding round.

Algorithm B can be implemented by two types of PEs as shown in Fig. 2(a). PE1s compute $\hat{\Delta}_{even}(x)$ and $\hat{\Theta}_{even}(x)$. PE0s update $\Lambda(x)$ and $B(x)$ and they are split into two rows to update the even and odd coefficients separately. Each PE is responsible for updating one pair of coefficients in the polynomials. Multiplications and divisions by x and x^2 are implemented by routing the coefficients to the adjacent or the second adjacent PEs, respectively. It can be observed that each group of one PE1 and two PE0s have 10 multipliers, 7 adders, 6 registers, and 3 multiplexers. The critical path is highlighted by the thicker wire in Fig. 2(a), and it consists of two multipliers and three adders. Similar to the nested RS KES architectures in [10], [12], each register in Fig. 2(a) can be replaced by two registers if the nested KES for two sub-codewords are interleaved according to the slow-down technique [11]. Then the registers can be moved around using the re-timing technique [11] to reduce the critical path to one multiplier and two adders as shown in Fig. 2(b). It should be noted that if the nested decoding only needs to be carried out on one sub-codeword, then a zero needs to be inserted after each data and every other clock cycle is spent on dummy computations when such a slowed-down architecture is used.

The overall architecture for the nested BCH KES is shown in Fig. 3. Since the maximum degree of $\Lambda(x)$ is t_v, each row of PE0s and PE1s has $h + 1 = \lceil (t_v + 1)/2 \rceil$ units. The degree of $B(x)$ may become higher than t_v. However, discarding the higher coefficients of $B(x)$ does not affect the $\Lambda(x)$ output by the nested KES when the errors are correctable. The registers outside of the PEs are introduced due to re-timing and initialization. In the beginning, the KES results from the sub-codeword decoding or the previous nested decoding round are loaded into the registers of the PEs, with the least significant coefficients in the left-most PEs. The coefficients

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 29,2026 at 14:11:03 UTC from IEEE Xplore. Restrictions apply.

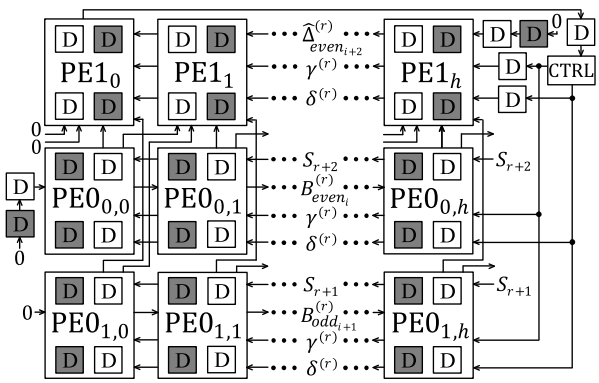


Fig. 3. Overall architecture of nested KES for GII-BCH decoding.

of the updated polynomials are also located at the left-most PEs at the end of every iteration.

B. Scaled Nested KES for GII-BCH Decoding

Finite field multipliers are much more expensive than adders. For example, a multiplier is more than 16 times larger than an adder over $GF(2^{12})$. Each multiplier in the PEs in Fig. 2 corresponds to a polynomial scaling operation in Algorithm B. As mentioned before, multiplications or divisions by powers of x are implemented by routing the coefficients to proper PEs and do not cause any logic complexity. The number of multipliers can be lowered by reducing the number of distinct product terms to compute. This goal was realized for nested RS KES by adopting scaled versions of the polynomials [12]. Comparing (7) and (5), the polynomial updating in the nested BCH KES involves more terms with more different scalars. Hence, it is much more challenging to enable the sharing of product terms in the nested BCH KES through scaling the polynomials and reformulating the algorithm.

Comparing the updating formulas in Lines 1 and 3 of Algorithm B, it is observed that the sum of the two terms $\gamma^{(r)} S_{r+2} \Lambda^{(r)}_{even}(x)$ and $\hat{\Delta}^{(r)}_0 S_{r+2} x^2 B^{(r)}_{even}(x)$ in Line 3 is $\Lambda^{(r+2)}_{even}(x)$ scaled by S_{r+2}. If the scaled version $\Lambda'^{(r+2)}_{even}(x) = S_{r+2}\Lambda^{(r+2)}_{even}(x)$ is used instead of $\Lambda^{(r+2)}_{even}(x)$, then the product terms in Line 3 can be shared to compute $\Lambda'^{(r+2)}_{even}(x)$ and two finite field multiplications are saved. Similarly, the sum of $\gamma^{(r)} S_{r+1} \Lambda^{(r)}_{odd}(x)/x$ and $\hat{\Delta}^{(r)}_0 S_{r+1} x B^{(r)}_{odd}(x)$ in Line 3 equals $S_{r+1}\Lambda^{(r+2)}_{odd}(x)/x$. Division by x is implemented by routing the coefficients one PE to the left in Fig. 3. Thus, if $\Lambda'^{(r+2)}_{odd}(x) = S_{r+1}\Lambda^{(r+2)}_{odd}(x)$ is computed instead in Line 2 of Algorithm B, another two multiplications can be eliminated. Scaled versions of $\hat{\Theta}^{(r+2)}_{even}(x)$ can be also obtained by sharing the product terms in Line 3 of Algorithm B. If Line 5 of Algorithm B is executed, $\hat{\Theta}'^{(r+2)}_{even}(x) = \gamma^{(r)}\hat{\Theta}^{(r+2)}_{even}(x)$ equals the sum of the first three terms in Line 3. Otherwise, $\hat{\Theta}'^{(r+2)}_{even}(x) = \hat{\Delta}^{(r)}_0 \hat{\Theta}^{(r+2)}_{even}(x)$ is the sum of the last three terms in Line 3. Scaled versions of $\Lambda^{(r)}_{even}(x)$ and $\Lambda^{(r)}_{odd}(x)$ can be used to update scaled $B^{(r+2)}_{even}(x)$ and $B^{(r+2)}_{odd}(x)$ as well.

If the number of errors does not exceed $w/2$, the error locations are the inverse roots of $\Lambda^{(w)}(x)$. The same error locations can be obtained as long as all the polynomials involved in the KES are scaled by the same factor. As analyzed in the previous paragraph, the polynomials need to be scaled by different factors in order to reuse the product terms to reduce the number of multipliers. To make up for the differences, additional factors need to be multiplied to make the overall scaling for every polynomial the same. The scalars for the r-th iteration are summarized in Table I. Depending on whether Lines 4-6 or 7-9 of Algorithm B are executed, there are two possibilities for the scalars, and the common overall scalars are $\gamma^{(r)} S_{r+1} S_{r+2}$ and $\hat{\Delta}^{(r)}_0 S_{r+1} S_{r+2}$, respectively. To avoid introducing additional multipliers, the additional scaling for iteration $r-2$ is combined with the scaling for iteration r.

TABLE I
SCALED POLYNOMIALS & ADDITIONAL SCALARS

polynomial	$\hat{\Delta}'^{(r+2)}_{even}(x)$	$\Lambda'^{(r+2)}_{odd}(x)$	$\Lambda'^{(r+2)}_{even}(x)$
scalar introduced for term sharing	1	S_{r+1}	S_{r+2}
additional scalar	$\gamma^{(r)} S_{r+1} S_{r+2}$ or $\hat{\Delta}^{(r)}_0 S_{r+1} S_{r+2}$	$\gamma^{(r)} S_{r+2}$ or $\hat{\Delta}^{(r)}_0 S_{r+2}$	$\gamma^{(r)} S_{r+1}$ or $\hat{\Delta}^{(r)}_0 S_{r+1}$
polynomial	$\hat{\Theta}'^{(r+2)}_{even}(x)$	$B'^{(r+2)}_{odd}(x)$	$B'^{(r+2)}_{even}(x)$
scalar introduced for term sharing	$\gamma^{(r)}$ or $\hat{\Delta}^{(r)}_0$	$\gamma^{(r)} S_{r+1}$ or $\hat{\Delta}^{(r)}_0 S_{r+1}$	$\gamma^{(r)} S_{r+2}$ or $\hat{\Delta}^{(r)}_0 S_{r+2}$
additional scalar	$S_{r+1} S_{r+2}$	S_{r+2}	S_{r+1}

Algorithm C Scaled Nested GII-BCH KES Algorithm

input: $\Lambda'^{(u)}_{even}(x)$, $\Lambda'^{(u)}_{odd}(x)$, $B'^{(u)}_{even}(x)$, $B'^{(u)}_{odd}(x)$, $\hat{\Delta}'^{(u)}_{even}(x)$, $\hat{\Theta}'^{(u)}_{even}(x)$ $\gamma'^{(u)}$, $k^{(u)}$ from previous KES; S_i $(u \le i < w)$; $S_w = 0$

initialization:

$\hat{\Delta}'^{(u)}_{even}(x) = \hat{\Delta}'^{(u)}_{even}(x) + S_u \Lambda'^{(u)}_{even}(x)$

$\hat{\Theta}'^{(u)}_{even}(x) = \hat{\Theta}'^{(u)}_{even}(x) + S_u B'^{(u)}_{even}(x)$

$\mathrm{I}_u = 1$ or S_{u-1}; $\mathrm{II}_u = S_{u+1}$; $\mathrm{III}_u = S_{u+2}$ or $S_{u-1}S_{u+2}$

for $r = u, u+2, \cdots, w-2$

1) $\gamma'^{(r)}_{\Lambda odd} = \gamma'^{(r)}\ \mathrm{II}_r$; $\gamma'^{(r)}_{\Lambda even} = \gamma'^{(r)}\ \mathrm{III}_r$; $\gamma'^{(r)}_{\Delta} = \gamma'^{(r)}\ \mathrm{I}_r$

2) $\hat{\Delta}'^{(r)}_{Bodd} = \hat{\Delta}'^{(r)}_0\ \mathrm{II}_r$; $\hat{\Delta}'^{(r)}_{Beven} = \hat{\Delta}'^{(r)}_0\ \mathrm{III}_r$; $\hat{\Delta}'^{(r)}_{\theta} = \hat{\Delta}'^{(r)}_0\ \mathrm{I}_r$

3) $\gamma'^{(r+2)} = \gamma'^{(r)}_{\Delta}\hat{\Delta}'^{(r)}_0$

4) $\mathrm{I}_{r+2} = S_{r+1}S_{r+2}$; $\mathrm{II}_{r+2} = S_{r+2}S_{r+3}$; $\mathrm{III}_{r+2} = S_{r+1}S_{r+4}$

5) $\Lambda'^{(r+2)}_{even}(x) = \gamma'^{(r)}_{\Lambda even}\Lambda'^{(r)}_{even}(x) + \hat{\Delta}'^{(r)}_{Beven} x^2 B'^{(r)}_{even}(x)$

6) $\Lambda'^{(r+2)}_{odd}(x) = \gamma'^{(r)}_{\Lambda odd}\Lambda'^{(r)}_{odd}(x) + \hat{\Delta}'^{(r)}_{Bodd} x^2 B'^{(r)}_{odd}(x)$

7) $\hat{\Delta}'^{(r+2)}_{even}(x) = \gamma'^{(r)}_{\Delta}\hat{\Delta}'^{(r)}_{even}(x)/x^2 + \gamma'^{(r)}_{\Lambda odd}\Lambda'^{(r)}_{odd}(x)/x + \gamma'^{(r)}_{\Lambda even}\Lambda'^{(r)}_{even}(x) + \hat{\Delta}'^{(r)}_{\theta}\hat{\Theta}'^{(r)}_{even}(x) + \hat{\Delta}'^{(r)}_{Bodd} x B'^{(r)}_{odd}(x) + \hat{\Delta}'^{(r)}_{Beven} x^2 B'^{(r)}_{even}(x)$

if $\hat{\Delta}'^{(r)}_0 \neq 0$ and $k^{(r)} \ge -1$

8) $B'^{(r+2)}_{even}(x) = \gamma'^{(r)}_{\Lambda even}\Lambda'^{(r)}_{even}(x)$

$B'^{(r+2)}_{odd}(x) = \gamma'^{(r)}_{\Lambda odd}\Lambda'^{(r)}_{odd}(x)$

9) $\hat{\Theta}'^{(r+2)}_{even}(x) = \gamma'^{(r)}_{\Delta}\hat{\Delta}'^{(r)}_{even}(x)/x^2 + \gamma'^{(r)}_{\Lambda odd}\Lambda'^{(r)}_{odd}(x)/x + \gamma'^{(r)}_{\Lambda even}\Lambda'^{(r)}_{even}(x)$

10) $k^{(r+2)} = -k^{(r)} - 2$

else

11) $B'^{(r+2)}_{even}(x) = \hat{\Delta}'^{(r)}_{Beven} x^2 B'^{(r)}_{even}(x)$

$B'^{(r+2)}_{odd}(x) = \hat{\Delta}'^{(r)}_{Bodd} x^2 B'^{(r)}_{odd}(x)$

12) $\hat{\Theta}'^{(r+2)}_{even}(x) = \hat{\Delta}'^{(r)}_{\theta}\hat{\Theta}'^{(r)}_{even}(x) + \hat{\Delta}'^{(r)}_{Bodd} x B'^{(r)}_{odd}(x) + \hat{\Delta}'^{(r)}_{Beven} x^2 B'^{(r)}_{even}(x)$

13) $k^{(r+2)} = k^{(r)} + 2$

Our proposed scaled nested KES algorithm for GII-BCH decoding is shown in Algorithm C. To differentiate from the notations used in Algorithm B, '′' is added to the polynomials

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 29,2026 at 14:11:03 UTC from IEEE Xplore. Restrictions apply.

and coefficients in Algorithm C. However, the inputs to both algorithms are the same and they are the results of the KES of the sub-codeword decoding or previous nested decoding round.

The scalar for each polynomial in iteration r of Algorithm C is contributed by three components: The additional scalar needs to be multiplied from iteration $r-2$, the scalar to be multiplied to enable product term sharing for iteration r, and the scalar originally on the polynomial in iteration r of Algorithm B. Take the scalar of $\Lambda'^{(r)}_{even}(x)$ in Line 5 of Algorithm C as an example. From Table I, if Lines 4-6 of Algorithm B were executed in iteration $r-2$, then the additional scalar from iteration $r-2$ for $\Lambda'^{(r)}_{even}(x)$ is $\gamma^{(r-2)}S_{r-1}$. Also the scalar introduced to enable the product term sharing in the computation of $\Lambda'^{(r+2)}_{even}(x)$ in iteration r is S_{r+2}. From Line 1 of Algorithm B, $\Lambda^{(r)}_{even}(x)$ is originally scaled by $\gamma^{(r)}$. Hence, the scalar of $\Lambda'^{(r)}_{even}(x)$ in Line 5 of Algorithm C should be $\gamma^{(r-2)}S_{r-1}S_{r+2}\gamma^{(r)}$. Different from that in the scaled nested RS KES, two syndromes instead of one are included in the products of scalars in the case of nested BCH KES due to the iteration skipping. As a result, one more multiplier is needed in the data path for computing the scalars. Fortunately, all the syndromes are available before the nested KES is carried out, and any computations on the syndromes can be carried out beforehand. From Table I, the possible combinations of the syndromes in the additional scalar parts of iteration r and the scalars for term sharing parts of iteration $r+2$ for the same polynomial are $S_{r+1}S_{r+2}$, $S_{r+2}S_{r+3}$, and $S_{r+1}S_{r+4}$. These products are pre-computed for iteration $r+2$ in Line 4 of Algorithm C to reduce the data path of scalar computations. Since they do not need to be computed in the last iteration of $r=w-2$, $S_{r+3}=S_{w+1}$ and $S_{r+4}=S_{w+2}$ in these formulas are not needed.

By replacing $S_{r-1}S_{r+2}$ with III_r, the scalar of $\Lambda'^{(r)}_{even}(x)$ when Lines 4-6 of Algorithm B were executed in the previous iteration becomes $\gamma^{(r-2)}\gamma^{(r)}\mathrm{III}_r$. Following the derivations in [12], it can be proved that $\gamma^{(r-2)}\gamma^{(r)} = \gamma'^{(r-2)}_{\Delta}\hat{\Delta}'^{(r-2)}_{0}$, which is denoted by $\gamma'^{(r)}$ as in Line 3 of Algorithm C. If Lines 7-9 were carried out in the previous iteration of Algorithm B, the scalar of $\Lambda'^{(r)}_{even}(x)$ would be $\mathrm{III}_r\hat{\Delta}^{(r-2)}_{0}\gamma^{(r)}$. $\hat{\Delta}^{(r-2)}_{0}\gamma^{(r)}$ also equals $\gamma'^{(r)}$ [12] in this case. Hence, the scalar of $\Lambda'^{(r)}_{even}(x)$ is always $\gamma'^{(r)}_{\Lambda_{even}} = \gamma'^{(r)}\mathrm{III}_r$. The scalars for the other polynomials can be derived as in Lines 1-2 of Algorithm C following derivations similar to those in [12].

The scalars in Table I are introduced to enable product term sharing. They do not originally belong to the nested KES algorithm. If any of the syndromes, $\gamma^{(r)}$, or $\hat{\Delta}^{(r)}_{0}$ is zero, then they should be replaced by '1's in the scalars in Table I to avoid mistakenly multiplying zeros to the polynomials. Similarly, if any of the reformulated scalars in Algorithm C introduced by our scaling are zero, they should be substituted by '1's. On the other hand, the scalars that are originally in Algorithm B should not be replaced by '1's even if they are '0's. For example, if $\hat{\Delta}^{(r)}_{0}=0$, then the $\hat{\Delta}'^{(r)}_{\theta}$, $\hat{\Delta}'^{(r)}_{Bodd}$, and $\hat{\Delta}'^{(r)}_{Beven}$ in Lines 11 and 12 of Algorithm C should be replaced by '1's since $\hat{\Delta}^{(r)}_{0}$ is not part of the original scalars in Lines 7 and 8 of Algorithm B. However, these scalars in Lines 5, 6, and 7 of Algorithm C should be zero since they are originally in Lines 1, 2, and 3 of Algorithm B.

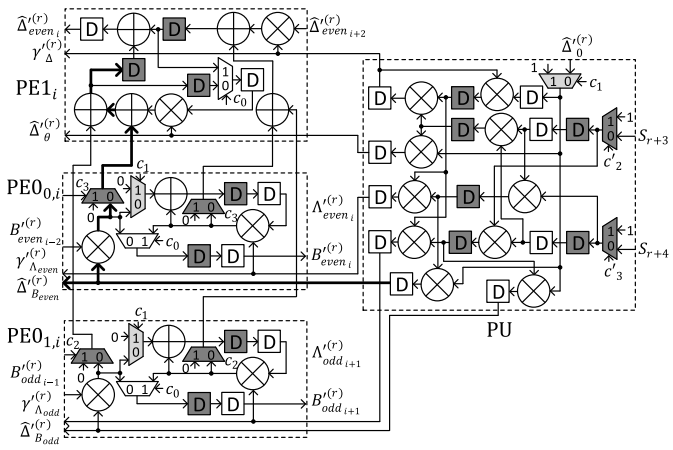


Fig. 4. PE architectures of scaled nested KES for GII-BCH decoding.

S_w is not available in the current nested decoding round and is set to zero in Algorithm C. Hence, the S_{r+2} in Table I for the last iteration of $r=w-2$ is replaced by '1'. Also $S_{r+1}=S_{w-1}$ in this iteration. Since the additional scaling for an iteration is put off until the next iteration in our scheme, the additional scaling for the last iteration is not carried out. Therefore, $\Lambda'^{(w)}_{odd}(x)$ and $B'^{(w)}_{odd}(x)$ are scaled by one more factor, S_{w-1}, compared to the other polynomials at the end of the last iteration. As a result, if Algorithm C starts from the results of a previous nested decoding round instead of the sub-codeword decoding, the other four polynomials need to be first scaled by S_{w-1}. This is achieved by multiplying an additional S_{u-1} to the initialization of I_u and III_u in Algorithm C, since u equals the w in the previous nested decoding round.

After the nested KES, the roots of $\Lambda'^{(w)}(x)$ are computed by the Chien search, which exhaustively evaluates the polynomial over $1, \alpha, \alpha^2, \cdots, \alpha^{n-1}$ to see if any of them are roots. Chien search architectures are available in [10]. $\Lambda'^{(w)}(x) = \Lambda'^{(w)}_{even}(x) + \Lambda'^{(w)}_{odd}(x)$. However, the $\Lambda'^{(w)}_{odd}(x)$ from the last nested KES iteration is scaled by S_{w-1} but $\Lambda'^{(w)}_{even}(x)$ is not. To find the roots of $\Lambda'^{(w)}(x)$, the Chien search can be done on $\Lambda'^{(w)}_{even}(x)$ and $\Lambda'^{(w)}_{odd}(x)$ separately. Then the evaluation values of $\Lambda'^{(w)}_{even}(x)$ are scaled by S_{w-1} before they are added up with the evaluation values of $\Lambda'^{(w)}_{odd}(x)$.

Our proposed scaled nested BCH KES algorithm can be implemented by the PE architectures illustrated in Fig. 4 after applying the slow-down and re-timing techniques. The overall architecture is similar to that in Fig. 3. $h+1=\lceil(t_v+1)/2\rceil$ PE1s and two rows of $h+1$ PE0s are needed. Since only 6 product terms are computed and shared to implement all polynomial updating in Algorithm C, each group of one PE1 and two PE0s in Fig. 4 only have 6 instead of 10 multipliers as needed by the PEs for the un-scaled Algorithm B in Fig. 2(b). Although multiplications are also required to compute the scalars in Lines 1-3 of Algorithm C, they are carried out on individual syndromes and scalars and are implemented by just one pre-processing unit (PU) as shown in Fig. 4. As a result, significant complexity reduction can be achieved by the scaled nested KES architecture. The critical path of the scaled architecture consists of one multiplier and three adders, which is only one more adder compared to that of the

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 29,2026 at 14:11:03 UTC from IEEE Xplore. Restrictions apply.

architecture in Fig. 2. The light and dark-gray multiplexers in Fig. 4 are employed to adjust the polynomial scaling when the syndromes, $\gamma^{(r)}$, and/or $\hat{\Delta}_0^{(r)}$ are zeros as discussed in previous paragraphs. The control signals c_2 and c_3 equal c_2' and c_3', respectively, delayed by one clock cycle.

C. Nested KES With Reduced Processing Elements

The techniques proposed in this section apply to both the scaled nested KES in Algorithm C and the nested KES in Algorithm B. To simplify the notations, the discussions below are based on the nested KES algorithm.

The number of PE0s in the nested KES architecture cannot be reduced since all coefficients of $\Lambda(x)$ are needed to compute the error locations. Moreover, the polynomials computed from the KES in nested decoding iteration i are needed to start the KES for nested decoding round $i+1$. Hence, enough PE1s need to be allocated to compute all coefficients of $\hat{\Delta}_{even}(x)$ up to the second to the last nested decoding round. However, in the last nested decoding round, the higher coefficients of $\hat{\Delta}_{even}(x)$ do not need to be computed or stored if they are not going to affect $\hat{\Delta}_0^{(r)}$ and hence the computations in any of the remaining nested KES iterations. Accordingly, some higher PE1s can be eliminated and we do not need all the $h+1 = \lceil (t_\upsilon+1)/2 \rceil$ PE1s as in Fig. 3.

Over the nested KES iterations, the length of $\hat{\Delta}_{even}(x)$ grows with those of $\Lambda_{even}(x)$ and $\Lambda_{odd}(x)$. Additionally, the coefficients of $\hat{\Delta}_{even}(x)$ are shifted to the left by one PE in each clock cycle, and the one at the leftmost PE is the discrepancy coefficient to be used in the next iteration. To simplify notations, let $t_\upsilon > t_{\upsilon-1}$. As long as the discrepancy coefficients in the $t_\upsilon - t_{\upsilon-1}$ iterations of the last-round nested KES are kept the same, the overall decoding output is not affected. From our simulations, L_Λ, which is the length of $\Lambda(x)$, increases by one every two iterations on average. Hence, assume that $L_\Lambda^{(2t_{\upsilon-1})} \le t_{\upsilon-1}+\sigma$ $(0 \le \sigma \le t_\upsilon - t_{\upsilon-1})$. Accordingly, $\lceil ((t_{\upsilon-1}+\sigma)+1)/2 \rceil$ PE1s are needed to hold the $\hat{\Delta}_{even}(x)$ coefficients up to the second to the last nested decoding round. Similarly, assume that $L_\Lambda^{(2t_{\upsilon-1}+4\rho)} \le t_{\upsilon-1}+2\rho+\sigma$. Having another $\lceil ((t_{\upsilon-1}+2\rho+\sigma)+1)/2 \rceil - \lceil (t_{\upsilon-1}+\sigma+1)/2 \rceil = \rho$ PE1s is sufficient to hold $\hat{\Delta}_{even}(x)$ for up to $4\rho/2 = 2\rho$ even iterations of the nested KES in the last round. These $\lceil (t_{\upsilon-1}+\sigma+1)/2 \rceil + \rho$ PE1s also provide valid discrepancy coefficients for another $\lceil (t_{\upsilon-1}+\sigma+1)/2 \rceil + \rho$ even nested KES iterations. Since there are $t_\upsilon - t_{\upsilon-1}$ even iterations in total in the last round, $2\rho + \lceil (t_{\upsilon-1}+\sigma+1)/2 \rceil + \rho$ must be at least $t_\upsilon - t_{\upsilon-1}$ to ensure that the discrepancy coefficients are correct for all iterations. In other words, the following constraint needs to be satisfied.

$$3\rho + \lceil (t_{\upsilon-1}+\sigma+1)/2 \rceil \ge t_\upsilon - t_{\upsilon-1} \tag{10}$$

For a given σ, ρ can be decided from (10). Then the nested KES can be implemented by the architecture in Fig. 5 with $l+1$ PE1s, where $l+1 = \lceil (t_{\upsilon-1}+\sigma+1)/2 \rceil + \rho$.

To determine the parameter σ, the probability of decoding failure brought by assuming $L_\Lambda^{(2t_{\upsilon-1})} \le t_{\upsilon-1}+\sigma$ needs to be estimated. Let $|e|$ be the number of errors of a corrupted sub-codeword. $L_\Lambda^{(r)}$ is upper bounded by $|e|$. Hence, only when $t_{\upsilon-1}+\sigma < |e|$, decoding failures may possibly result from

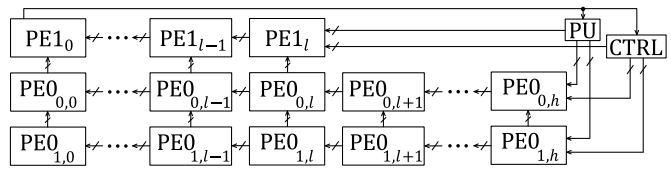


Fig. 5. Nested BCH KES architecture with reduced PEs.

assuming $L_\Lambda^{(2t_{\upsilon-1})} \le t_{\upsilon-1}+\sigma$. Also the errors are not correctable anyway by the GII code if $|e| > t_\upsilon$. Therefore, extra decoding failures may happen when there is a sub-codeword with $t_{\upsilon-1}+\sigma < |e| \le t_\upsilon$.

Even if $|e| > t_{\upsilon-1}+\sigma$, $L_\Lambda^{(2t_{\upsilon-1})}$ may still do not exceed $t_{\upsilon-1}+\sigma$. Next, the conditions under which $L_\Lambda^{(2t_{\upsilon-1})} > t_{\upsilon-1}+\sigma$ are derived. L_Λ is non-decreasing over the iterations. If $L_\Lambda^{(r)} > t_{\upsilon-1}+\sigma$ first occurs at $r = r^* \le 2t_{\upsilon-1}$, then $L_\Lambda^{(2t_{\upsilon-1})} > t_{\upsilon-1}+\sigma$.

Lemma 1 If $L_\Lambda^{(r)} > t_{\upsilon-1}+\sigma$ first occurs at $r = r^* \le 2t_{\upsilon-1}$, then iterations $r^*-2-2i^*, \cdots, r^*-2-4, r^*-2-2$ have zero discrepancy coefficients, where $i^* = (t_{\upsilon-1} - r^*/2) + \sigma + 1$.

Proof: Denote the length of $B^{(r)}(x)$ by $L_B^{(r)}$. This lemma can be proved by using two properties: i) $L_\Lambda^{(r)} + L_B^{(r)} = r-1$ [7]; ii) if the conditions for the 'if' statement in Algorithm B are satisfied (Lines 4-6 are executed) at iteration $r-2$, the updated polynomials satisfy $L_\Lambda^{(r)} \ge L_B^{(r)} + 1$. The second property can be easily derived by following the computations in Algorithm B.

From Lines 1-2 of Algorithm B, L_Λ only increases if the length of $x^2B(x)$ is larger. If $L_\Lambda^{(r)} > t_{\upsilon-1}+\sigma$ first occurs at $r = r^*$, then the length of $x^2B^{(r^*-2)}(x)$ is at least $t_{\upsilon-1}+\sigma+1$ and hence $L_B^{(r^*-2)} \ge t_{\upsilon-1}+\sigma-1$. From the first property, it can be derived that $L_\Lambda^{(r^*-2)} \le r^*-2-t_{\upsilon-1}-\sigma$. Suppose that Lines 7-9 of Algorithm B were executed (the 'if' conditions were not satisfied) for i even consecutive iterations prior to iteration r^*-2 and Lines 4-6 were executed (the 'if' conditions were satisfied) in the iteration before. In each of those i iterations, L_B increases by 2 and L_Λ does not change. Thus, $L_B^{(r^*-2-2i)} = L_B^{(r^*-2)} - 2i \ge t_{\upsilon-1}+\sigma-1-2i$ and $L_\Lambda^{(r^*-2-2i)} = L_\Lambda^{(r^*-2)} \le r^*-2-t_{\upsilon-1}-\sigma$. From the second property, after Lines 4-6 of Algorithm B were executed at iteration $r^*-2-2i-2$, $L_\Lambda^{(r^*-2-2i)} - L_B^{(r^*-2-2i)} \ge 1$ holds. Accordingly, $i > i^* = (t_{\upsilon-1} - r^*/2) + \sigma + 1$.

In each of those i^* iterations $(r^*-2-2i^* \le r \le r^*-4)$, $L_\Lambda^{(r)} = L_\Lambda^{(r^*-2)} \le r^*-2-t_{\upsilon-1}-\sigma$. On the other hand, $L_B^{(r)} = L_B^{(r^*-2)} - (r^*-2-r) \ge t_{\upsilon-1}+\sigma-1-(r^*-2-r)$. The $k^{(r)}$ in Algorithm B actually equals $L_B^{(r)} - L_\Lambda^{(r)}$. Utilizing $i^* = (t_{\upsilon-1} - r^*/2) + \sigma + 1$, it can be easily derived that $L_B^{(r)} - L_\Lambda^{(r)} \ge -1$. Therefore, in those i^* iterations, the 'if' conditions are not satisfied because $\hat{\Delta}^{(r)}(x) = 0$. □

Since $r^* \le 2t_{\upsilon-1}$, $i^* \ge \sigma+1$. From Lemma 1, at least $\sigma+1$ even consecutive iterations with zero discrepancy coefficients should have happened to make $L_\Lambda^{(2t_{\upsilon-1})} > t_{\upsilon-1}+\sigma$, which can be generalized to $L_\Lambda^{(2r)} > r+\sigma$. For random input errors, the discrepancy coefficients are independent and their values are uniformly distributed over $GF(2^q)$. Therefore, the probability of $L_\Lambda^{(2t_{\upsilon-1})} > t_{\upsilon-1}+\sigma$ can be estimated as $2^{-(\sigma+1)q}$. The probability of $L_\Lambda^{(2t_{\upsilon-1}+4\rho)} > t_{\upsilon-1}+2\rho+\sigma$

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 29,2026 at 14:11:03 UTC from IEEE Xplore. Restrictions apply.

is the same. However, the chance that a sub-codeword has $|e| > t_{\upsilon-1} + \sigma + \rho$ errors with $\rho \geq 1$ is even smaller. Hence the contribution of the assumption $L_{\Lambda}^{(2t_{\upsilon-1}+4\rho)} > t_{\upsilon-1} + 2\rho + \sigma$ to the extra decoding failure is ignored. As analyzed previously, extra decoding failures only happen when there is a sub-codeword with $t_{\upsilon-1} + \sigma < |e| \leq t_\upsilon$ and all the other sub-codewords are correctable. As a result, the probability of extra decoding failure brought by using $\lceil (t_{\upsilon-1}+\sigma+1)/2 \rceil + \rho$ PE1 can be estimated as

$$\binom{m}{1} P_1 (P_2)^{m-1} 2^{-(\sigma+1)q}, \tag{11}$$

where $P_1 = \sum_{j=t_{\upsilon-1}+\sigma+1}^{t_\upsilon} \binom{n}{j} (p_b)^j (1-p_b)^{n-j}$ and $P_2 = \sum_{j=0}^{t_{\upsilon-1}} \binom{n}{j} (p_b)^j (1-p_b)^{n-j}$ are the probabilities that there are $t_{\upsilon-1} + \sigma < |e| \leq t_\upsilon$ and $0 \leq |e| \leq t_{\upsilon-1}$ errors in a sub-codeword, respectively, and p_b is the input BER. $(P_2)^{m-1}$ is a loose upper bound of the probability that the other $m-1$ sub-codewords are correctable. Similar analyses can be extended to the cases that the t_i's are not distinct.

The parameter σ should be chosen so that the resulted FER degradation is negligible. Typically, a very small σ is sufficient and the extra FER decreases very fast for larger σ. Take the GII-BCH codes whose FERs are shown in Fig. 1 as an example. For the code with $\upsilon = 3$, when $\sigma = 2$, the upper bound of the extra decoding failure rate at BER $= 2.5 \times 10^{-3}$ computed from (11) is 2.2×10^{-24}. It is negligible compared to the FER of this code at this BER. When σ is increased to 3, the extra decoding failure rate is further reduced to 1.2×10^{-28}. It has been found in [5], [10] as well as the simulations in this work that, for a given amount of redundancy, the GII codes achieving the best error-correcting performance typically have t_υ much larger than $t_{\upsilon-1}$. As a result, significant area saving can be achieved by eliminating the higher PE1s.

IV. Complexity Analyses and Comparisons

Using a GII-BCH code over $GF(2^{12})$ with $m = 8$ and $\upsilon = 3$ as an example, the complexities of our proposed nested KES architectures are analyzed and compared in this section. The FER of this code is shown in Fig. 1 and $[t_0, t_1, t_2, t_3] = [28, 32, 39, 58]$ is adopted in the decoder design. The code rate is 90.2% and the overall codeword length is around 4kB. This code is a good candidate for Flash memory applications.

For the nested GII-BCH KES algorithm (Algorithm B), the slowed-down and re-timed architectures for the PEs are shown in Fig. 2(b). Each group of one PE1 and two PE0s have 12 registers, 10 multipliers, 7 adders, and 3 multiplexers. The critical path consists of one multiplier and two adders. From Fig. 3, the overall nested KES architecture has $h + 1 = \lceil (t_\upsilon + 1)/2 \rceil = 30$ groups of such PEs. Considering that there are 7 registers outside of the PEs, the total numbers of hardware units needed for the nested BCH KES architecture are listed in Table II. The control logic is simple and accounts for a negligible portion of the overall area. Hence, its complexity is not included in the table.

Each adder over $GF(2^{12})$ is implemented as 12 bit-wise XOR and a multiplexer has similar complexity. A 12-bit register with preset and clear can be implemented by the area of around 36 XORs. The complexity of a $GF(2^{12})$ multiplier is reduced by carrying out the arithmetic over composite field $GF(((2^3)^2)^2)$. This field can be constructed iteratively from $GF(2)$ using polynomials x^3+x+1, $x^2+\mu$, and $x^2+\varphi$ that are irreducible over $GF(2)$, $GF(2^3)$ and $GF((2^3)^2)$, respectively. This composite field multiplier is implementable by the area of around 201 XOR gates, and it has 9 gates in the critical path. Using the complexities of these components, the total gate count and critical path of the nested BCH KES architecture are calculated as shown in Table II.

TABLE II

Hardware Complexities of Nested BCH KES Architectures

	Adder	Mult.	Mux	Reg.	Total (# XORs)	crit. path (# gates)
Approach from [9]	832	892	60	120	194316	15
Nested KES (Algorithm B)	210	300	90	367	77112	11
Scaled nested KES (Algorithm C)	210	190	273	410	57648	12
Scaled nested KES with reduced PEs	165	172	228	365	51546	12

For the scaled nested GII-BCH KES algorithm (Algorithm C), the number of PEs is the same as that in Fig. 3 and there are still $h + 1 = 30$ groups of three PEs. However, the number of multipliers in each group is reduced from 10 to 6 as shown in Fig. 4. Although there are one more register and a larger number of multiplexers for adjusting the zero scalars in the PEs, their area is much smaller than that of multipliers. Moreover, each of those multiplexers with a '0' or '1' input can be simplified to bit-wise AND or OR gates, whose complexity can be considered as half of the multiplexer. Although the PU for scalar computation brings area overheads, only one PU is needed. As a result, the scaled nested architecture in Fig. 4 achieves $1 - 57648/77112 = 25\%$ area reduction over that in Fig. 2 at the cost of one more gate in the critical path.

From (10), it can be derived that $3\rho + 21 \geq 19$ needs to be satisfied in order not to affect the decoding result. Hence, $\rho = 0$ is sufficient. This means that the minimum number of PE1s needed for the example code is $\lceil (t_{\upsilon-1}+1)/2 \rceil + 1 = 21$. Compared to the $h + 1 = 30$ PE1s needed in the nested KES and scaled nested KES architectures, significant complexity reduction is achieved. Additionally, the outputs of the four dark gray multiplexers in the pair of PE0s of the nested KES architecture in Fig. 4 are only used in PE1. Since the 9 higher PE1s are eliminated, these multiplexers in the 9 higher pairs of PE0s can be also removed. Overall, the scaled nested KES architecture with eliminated higher PE1s and simplified higher PE0s achieves $1 - 51546/77112 = 33\%$ area reduction.

For comparison, the complexity of the nested KES using the approach in [9] is also listed in Table II. The design in [9] employs m riBM decoders capable of correcting t_υ errors each and the KES in these decoders is shared for individual sub-codeword decoding and nested decoding. However, the majority of the KES PEs in this design will be idling most of the time since the nested decoding is activated with low probability. To compare with our design, the complexity of only the $2(t_\upsilon - t_0)$ extra PEs caused by the nested KES for one decoder is included in Table II. The design in [9] also adopts a multiplier-adder tree to reinitialize each discrepancy coefficient at the beginning of the nested KES. These trees require a large number of multipliers and lead

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 29,2026 at 14:11:03 UTC from IEEE Xplore. Restrictions apply.

to longer critical path. Our best nested KES only requires 51546/194316 = 26.5% of the area and has much shorter critical path.

The number of clock cycles needed by each of the designs in Table II is the same, which is $t_i - t_{i-1}$ clock cycles for the i-th nested decoding round. Therefore, the latency of the designs are proportional to the critical path.

Compared to our nested KES architecture, our scaled architecture achieves area saving by reducing the number of multipliers in the PEs. The complexity of finite field multipliers increases more than linearly with the order of the field. For codes constructed over higher-order finite fields, the multipliers occupy a larger portion of the overall area. Hence, the area reduction that is achievable by the scaled nested KES architecture would be more significant. The third design in this paper achieves further area reduction by eliminating higher PE1s. Ignoring the ceiling function in (10) and also considering that ρ should be non-negative, it can be computed that the minimum ρ is around $\max\{0, t_\upsilon/3 - t_{\upsilon-1}/2\}$. Hence, the minimum number of PE1s needed is around $t_{\upsilon-1}/2 + \max\{0, t_\upsilon/3 - t_{\upsilon-1}/2\} = \max\{t_\upsilon/3, t_{\upsilon-1}/2\}$. Accordingly, the ratio of PE1 reduction is around 1/3 when $t_{\upsilon-1}/t_\upsilon \leq 2/3$ and is $1 - t_{\upsilon-1}/t_\upsilon < 1/3$ otherwise. For a variety of GII codes, $t_\upsilon/3$ and $t_{\upsilon-1}/2$ are quite close in the settings optimized for error-correcting capability [5]. Therefore, we expect that around 1/3 of the PE1s can be eliminated in the nested KES for various codes.

V. Conclusions

In this paper, three designs have been proposed to allow every other iteration to be skipped in the nested KES for binary GII-BCH decoding without suffering from the clock frequency bottleneck. The challenges brought by incorporating two higher-order syndromes in one nested KES iteration are addressed by algorithmic reformulations and architectural modifications. The scaled nested KES architecture allows substantial complexity reduction by enabling the sharing of product terms. Additionally, the number of PEs is reduced by a significant portion without affecting the decoding output through analyzing the data dependencies over the nested KES iterations. Future work will study the implementation of the other components of GII-BCH decoders.

References

[1] X. Tang and R. Koetter, "A novel method for combining algebraic decoding and iterative processing," in *Proc. IEEE Intl. Symp. Inf. Theory*, Seattle, WA, USA, Jul. 2006, pp. 474–478.

[2] X. Zhang, "Modified generalized integrated interleaved codes for local erasure recovery," *IEEE Commun. Lett.*, vol. 21, no. 6, pp. 1241–1244, Jun. 2017.

[3] X. Zhang, "Generalized three-layer integrated interleaved codes," *IEEE Commun. Lett.*, vol. 22, no. 3, pp. 442–445, Mar. 2018.

[4] X. Zhang, "Systematic encoder of generalized three-layer integrated interleaved codes," in *Proc. IEEE Intl. Conf. Commun.*, Shanghai, China, May 2019, pp. 1–6.

[5] Y. Wu, "Generalized integrated interleaved codes," *IEEE Trans. Inf. Theory*, vol. 63, no. 2, pp. 1102–1119, Feb. 2017.

[6] X. Zhang, "Decoding of generalized three-layer integrated interleaved codes," in *Proc. IEEE Intl. Symp. Inf. Theory*, Paris, France, Jul. 2019, pp. 2424–2428.

[7] E. R. Berlekamp, *Algebraic Coding Theory (Revised Edition)*. Laguna Hills, CA, USA: Aegean Park, 1984.

[8] D. V. Sarwate and N. R. Shanbhag, "High-speed architectures for Reed-Solomon decoders," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 9, no. 5, pp. 641–655, Oct. 2001.

[9] W. Li, J. Lin, and Z. Wang, "A 124-Gb/s decoder for generalized integrated interleaved codes," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 66, no. 8, pp. 3174–3187, Aug. 2019.

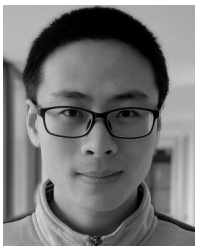
[10] X. Zhang and Z. Xie, "Efficient architectures for generalized integrated interleaved decoder," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 66, no. 10, pp. 4018–4031, Oct. 2019.

[11] K. K. Parhi, *VLSI Digital Signal Processing Systems: Design and Implementation*. Hoboken, NJ, USA: Wiley, 1999.

[12] Z. Xie and X. Zhang, "Scaled nested key equation solver for generalized integrated interleaved decoder," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, early access, Feb. 3, 2020, doi: 10.1109/TCSII.2020.2970970.

[13] X. Zhang, *VLSI Architectures for Modern Error-Correcting Codes*. Boca Raton, FL, USA: CRC Press, 2015.

[14] W. Liu, J. Rho, and W. Sung, "Low-power high-throughput BCH error correction VLSI design for multi-level cell NAND flash memories," in *Proc. IEEE Workshop Signal Process. Syst.*, Oct. 2006, pp. 303–308.



Zhenshan Xie received the B.S. degree in information engineering from the East China University of Science and Technology, Shanghai, China, in 2014, and the M.S. degree in communications and information system from the University of Chinese Academy of Sciences, Beijing, China, in 2017. He is currently pursuing the Ph.D. degree with the Department of Electrical and Computer Engineering, The Ohio State University, Columbus, OH, USA.

His current research focuses on the design of high-performance very-large-scale integration (VLSI) architectures for error-correcting codes.

Xinmiao Zhang (Senior Member, IEEE) received the Ph.D. degree in electrical engineering from the University of Minnesota.

She joined The Ohio State University as an Associate Professor in 2017. Prior to that, she was a Timothy E. and Allison L. Schroeder Assistant Professor from 2005 to 2010 and Associate Professor from 2010 to 2013 at Case Western Reserve University. Between her academic positions, she was a Senior Technologist at Western Digital/SanDisk Corporation. She authored the book *VLSI Architectures for Modern Error-Correcting Codes* (CRC Press, 2015), and co-edited *Wireless Security and Cryptography: Specifications and Implementations* (CRC Press, 2007). Her research spans the areas of VLSI architecture design, digital storage and communications, security, and signal processing.

Dr. Zhang received an NSF CAREER Award in January 2009. She is also the recipient of the Best Paper Award at 2004 ACM Great Lakes Symposium on VLSI and 2016 International SanDisk Technology Conference. She was elected to serve on the Board of Governers of the IEEE Circuits and Systems Society for the 2019–2021 term. She is a Co-Chair of the Data Storage Technical Committee (2017–2020), and a member of the CASCOM and VSA technical committees and DISPS technical committee advisory board of IEEE. She served on the technical program and organization committees of many conferences, including ISCAS, SiPS, ICC, GLOBECOM, GlobalSIP, and GLSVLSI. She has been an Associate Editor for the IEEE Transactions on Circuits and Systems-I 2010-2019 and IEEE Open Journal of Circuits and Systems since 2019.

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 29,2026 at 14:11:03 UTC from IEEE Xplore. Restrictions apply.